

HTI.320 Group project final report

Group 7: Donya Davoodi, Jere Kivikangas, Elias Peltonen, Faezeh Sayad Sijani, Prajjwol Tripathi

Date: 3.12.2025

Source code and schematics: <https://github.com/3liasP/simdash>

Introduction

In today's world, the combination of hardware and software has made it possible to create simulations and recreate real experiences. The car simulator project can be used in various fields, for example, driving training, or even in medicine and psychology to analyze people's behavior during the driving process, or to automatically control cars in times when the driver may have difficulty, so this project can be a basis for more advanced projects, and we in the group decided to work on its development.

Prototype and vision

The present project is a set of Arduino Uno components, a digital display to display speed, and a small fan to simulate wind flow. Initially, our goal was to analyze driver data using a heart rate sensor; this has various advantages. For example, when the driver is stressed and his heart rate is out of normal, the car can be automatically slowed down, or it can even be used to examine the impact of various factors on the driver's stress while driving.

In the software section, the main goal is to design and implement a realistic environment for driving simulation and will include the following:

- Creating a driving scene: Designing a graphical environment with different paths.
- Implementing application physics: Applying physical laws such as friction and collision.
- Adding a speed signal: Displaying speed and its relationship with other system components.
- High-speed warning system: Creating a warning mechanism to prevent the risk of high-speed driving.
- Simulating weather conditions: Adding rainy, snowy, and sunny modes to increase realism.
- Random scenarios: Designing unexpected events to increase the challenge of realism.
- Testing and feedback: Testing the system, receiving feedback, and improving quality to make the driving experience more realistic.

Design of the prototype and vision

Due to the absence of a heart rate sensor, the hardware uses a steering wheel, a digital display to display speed, and a small fan to simulate wind flow. These components are seamlessly connected to the Arduino board to provide tangible outputs based on user performance and environmental data processing.

In the software section, the driving environment is designed with the ability to control the direction by the steering wheel of the car, simulate weather conditions, sound, and apply the laws of physics. The system is also able to display a speed signal and send a warning signal at high speeds. In addition, random scenarios such as animals entering the road are simulated.

The overall vision of this prototype is based on the combination of real-world simulation data. The user not only interacts with the system through the steering wheel and pedals but also experiences the output (via the fan). This simulator gives the user the opportunity to experience driving in a simulated environment.

Compare proposal with final project

As mentioned above, in the initial document, we decided to use a heart rate sensor and examine the impact of environmental factors and sudden scenarios on the driver's behavior. Even before that, we decided to use ECG helmets to retrieve brain signals and examine them. For the first part, which was related to the heart rate sensor, because this sensor did not reach us on time and we were worried that we would not be able to manage it by the time of project presentation, it was removed from our final design. The use of ECG helmets and brain signals, in addition to the fact that its parts were not available, required a lot of time to implement, while being attractive and extensive. By removing these two parts, the volume of the hardware section was reduced. And the current output of our group, a driving simulator using a steering wheel, allows the driver to control the car in a virtual environment. The driver can experience driving in an environment such as rain; while driving with the wind produced by a fan, he can feel the wind flow. Our final version is, however, more than satisfactory for the scope of this course, since it includes two hardware elements, the speedometer and fan, controlled Arduino board connected to Unity.

Development stages

Connecting Arduino to Unity:

- Using libraries to establish serial communication between Arduino and Unity.
- Receiving data from Unity and sending feedback based on the received data.

Software stages in Unity:

- Designing 3D environments, paths, obstacles, and weather conditions.
- Applying laws of physics such as friction, collision and the effect of environmental factors.
- Synchronizing the connection between software data and hardware data.
- Displaying warning messages in certain conditions.

Using GitHub for project management:

- Creating a common repository for Arduino and Unity source code
- Creating different branches for features keeping the main branch as stable as possible
- Merging these branches and checking the final output

Existing challenges:

- Sync changes made by group members
- Language barriers and team communication
- Not much pre-existing experience in used technologies as a team
- Busy schedules that don't allow many more meetings other than during the course lectures

Prototype interaction and testing

Currently, the final version is pretty much in line with the goal we set, and we are trying to test it to produce the best version by the deadline. As mentioned above, we had to ditch the heart rate sensor section due to time constraints, since we didn't receive it on time. However, skipping the heart sensor allowed us to polish other areas further, for example by improving driving scene and effects.

Lessons learned

Setting clear milestones and vision early on helped us a lot in the beginning. As a group, we could've benefited from better and clearer communication and participation. In the end, we learned a lot from the technologies used and the overall process of rapid prototyping. Additionally, we were able to produce a complete working prototype that matches the original vision without too many changes.

Donya

At the beginning, the project was honestly pretty hard for me because the others had more experience with Unity and the technical side, and I wasn't really sure where to start. I mostly just did whatever tasks came to my mind or whatever I felt I could handle, since our task division wasn't very clear and didn't work out perfectly.

Step by step, I pushed myself to learn and tried to contribute however I could. I learned the basics of Unity well enough to actually support the team, and I realized I really enjoy working with it now. GitHub was another big challenge for me, since I had never used it before, but with the help of my teammates I managed to understand it. I'm actually happy I got the chance to learn it, and I want to keep getting better.

Overall, the beginning was stressful, but little by little I learned a lot, added what I could to the project, and it turned into a really valuable experience for me.

Jere

I think the milestones were a good starting point for the project, but it was difficult to plan far into the future, and we had to change some of these since we didn't get the hardware we were planning to use. We also didn't divide the tasks really, and everyone did what they "wanted" or were able to, which worked but also created a bit of chaos when it came to merging the products of each member. We could've also benefited from a better scheduling of features, but with a short amount of time I think we succeeded in building the prototype pretty well.

Elias

I would try to establish an understanding of the group's skill set early on and try to divide tasks more efficiently. When conducting rapid prototyping, there is a fine line between "being agile and breaking things" and not reading documentation or following tutorials properly. At first, I ended up doing most of the groundwork, but in the end, everyone contributed, and the final result is something we can all be proud of. The group work sessions went by quickly and especially sharing hardware-related tasks was challenging (almost impossible) since most of us worked remotely outside of class. Looking back, we could have planned our session agendas beforehand, as we sometimes spent more time coordinating than actually making progress on the project. All in all, I think we succeeded fairly well and everyone definitely learned something new.

Faezeh

It was definitely a good experience for me, I would have liked the part related to manufacturing and hardware to be more, it was easier for me and my expertise to work in that part, I even thought about controlling the speed of a car when the heart rate was high and it was interesting for me, definitely in a group not everyone can be at the same level, I am grateful to my teammates, I was able to learn from them. In addition, I really had a lot of problems with my system which gave me a big challenge, however I tried my best.

Prajwol

Early use of Miro and Github was a good start for the team so everyone could see what needed to be done. We had freedom to do whatever task we wanted to do which was good but sometimes it some team members had confusions what to do which could have been easier if we had divided the task but pros and cons are always there some enjoy freedom some enjoy order. Overall the project work was successful and went well and most importantly everyone in team learned something new.